

Nexus Agent

Toàn cảnh hệ thống

Từ một câu hỏi trên trình duyệt đến điều phối đa agent, OpenBB/Qlib, phê duyệt của con người, bằng chứng, dữ liệu bền vững và các ranh giới bảo mật — giải thích bằng tiếng Việt với ví dụ dễ theo dõi.

1+4

Graph mặc định +
standalone Studio opt-in

4

Subagent chuyên trách và
tách quyền rõ ràng

3

Worker nội bộ cho dữ liệu,
Qlib và AI-Trader

1

PostgreSQL làm nguồn sự
thật của product state

2

Profile: local development
và private beta một node

∞

Dòng bằng chứng nối
request → dữ liệu → kết
quả

Cách đọc tài liệu này

Đây là bản đồ tổng thể của repository **Nexus Agent**, dành cho cả người mới làm quen lẫn người cần truy vết sâu đến file source. Mỗi chương đi theo ba lớp: hệ thống làm gì, vì sao thiết kế như vậy, và có thể kiểm chứng ở đâu trong mã nguồn.

Nếu bạn mới bắt đầu

Đọc chương 1 → 4, sau đó xem ba ví dụ ở chương 18. Bạn sẽ hiểu sản phẩm, các thành phần chính và cách một yêu cầu chạy xuyên hệ thống.

Nếu bạn là developer

Đọc chương 5 → 17 và phụ lục. Các bảng chỉ rõ entrypoint, contract, route, quyền, storage owner và lệnh kiểm tra.

Nếu bạn vận hành beta

Tập trung vào chương 14 → 17: profile triển khai, bảo mật, quan sát, migration, failure mode và quality gate.

Nếu bạn đánh giá kết quả quant

Đọc chương 10 → 13 để hiểu identity, lineage, giới hạn point-in-time, label lookahead, Evidence UI và ý nghĩa metric.

Phạm vi phải nhớ: hệ thống hiện chứng minh cho phát triển local và *một* private-beta node. Nó chưa tuyên bố hỗ trợ multi-node, disaster recovery ngoài máy, object storage từ xa, signed release, formal SLO hay capacity certification.

Quy ước trong tài liệu

Ký hiệu	Ý nghĩa	Ví dụ
<code>đường/dẫn</code>	File hoặc thư mục trong repository	<code>backend/source/http_app.py</code>
<code>Public</code>	Đi qua browser/backend, có auth theo profile	<code>GET /v1/topology</code>
<code>Internal</code>	Hợp đồng backend ↔ worker, không phải API cho browser	<code>POST /v1/experiments</code> của quant-worker
Authority	Nơi có quyền quyết định trạng thái chính thức	PostgreSQL cho workflow và approval
Artifact bytes	File lớn nằm trên filesystem, được bảo vệ bằng manifest/hash	Output của Qlib dưới <code>data/artifacts</code>

Nguồn kiểm chứng chính: source code và executable configuration tại commit `04aa3fe0302b` . Các file Markdown trong `docs/` được dùng làm bản đồ điều hướng, không thay thế source.

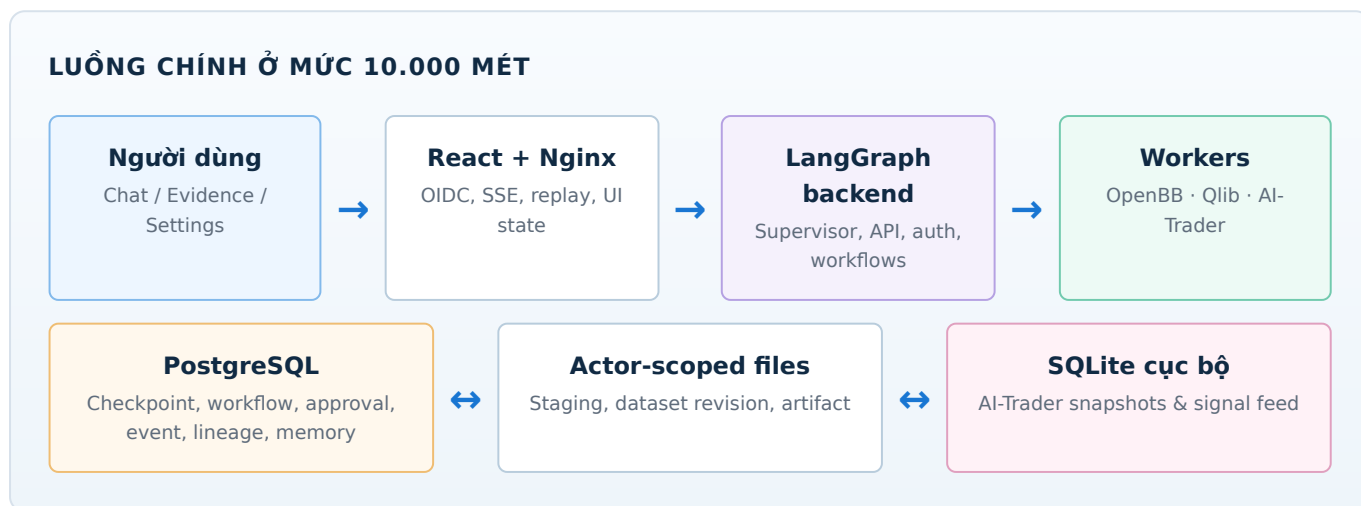
Mục lục

01	Tóm tắt điều hành	02	Bài toán, giá trị và phạm vi
03	Bản đồ repository & công nghệ	04	Kiến trúc tổng thể
05	Frontend và trải nghiệm người dùng	06	Backend, LangGraph và streaming
07	Agent harness và phân quyền nhiệm vụ	08	Contract, event và tính đúng đắn
09	PostgreSQL, filesystem và ownership	10	Workflow OpenBB → Qlib dataset
11	Workflow Qlib experiment	12	AI-Trader integration
13	Evidence, dossier và lineage	14	Bảo mật, approval và user memory
15	Khởi chạy và triển khai	16	Quan sát, lỗi và phục hồi
17	Kiểm thử và quality gate	18	Ví dụ end-to-end để hiểu
19	Hướng dẫn đọc và thay đổi source	20	Giới hạn hiện tại và hướng mở rộng
A	Danh mục API	B	Cấu hình quan trọng
C	Glossary	D	Checklist vận hành

Đường tắt 15 phút: đọc sơ đồ chương 4, bảng agent ở chương 7, hai state machine ở chương 10-11, rồi chạy qua ví dụ 18.1. Đây là “xương sống” của toàn hệ thống.

01 Tóm tắt điều hành

Nexus Agent là một hệ thống xác thực sản phẩm cho nghiên cứu có nguồn dẫn, chuẩn bị dữ liệu tài chính OpenBB, thực nghiệm Qlib và chia sẻ bằng chứng qua AI-Trader. Một supervisor dùng LangGraph/Deep Agents điều phối bốn agent chuyên trách; React hiển thị chat, approval, execution graph và Evidence UI.



Năm ý quan trọng nhất

- Model không sở hữu quy trình nghiệp vụ.** Agent chỉ gọi tool cấp cao. Trusted Python code kiểm tra input, identity, trạng thái, idempotency, retry và postcondition.
- Dữ liệu và kết quả có danh tính bất biến.** Staging revision, dataset revision, experiment và artifact được nối bằng hash/manifest/lineage; alias chỉ là nhãn dễ đọc.
- Side effect nhạy cảm cần con người.** Lưu/xóa memory và publish strategy/discussion gây LangGraph interrupt; approval được ghi bền vững và ràng buộc với đúng action digest.
- Frontend dựng trạng thái từ protocol thật.** SSE, product-event replay và `GET /v1/topology` điều khiển chat, graph activity, approval và khả năng phục hồi sau reload.
- Đây là research evidence, không phải lời khuyên đầu tư.** Dữ liệu yfinance thiếu bảo đảm point-in-time, membership lịch sử, revision và corporate-action semantics đầy đủ.

React

Console phía browser; state phiên làm việc không phải product authority.

Python

Backend và ba worker FastAPI, chia theo contract HTTP.

Postgres

16

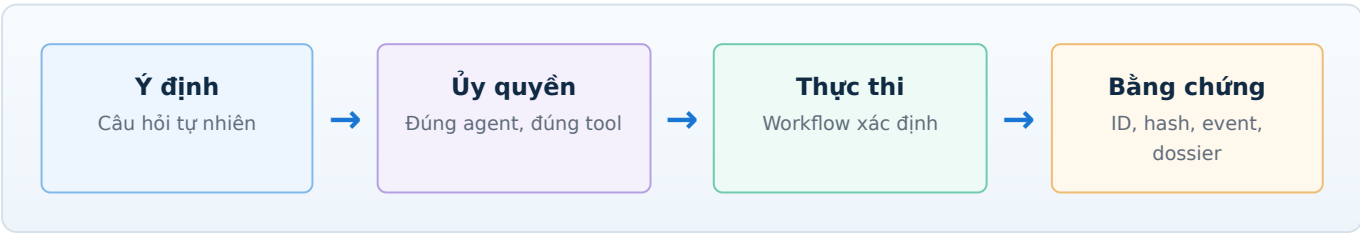
Authority cho state bền vững và actor isolation.

02 Bài toán, giá trị và phạm vi

Bài toán cốt lõi

Một quy trình nghiên cứu định lượng thực tế không chỉ là “hỏi LLM rồi nhận câu trả lời”. Nó phải biết nguồn nào được dùng, dữ liệu nào đã được tải, phiên bản dataset nào đi vào thử nghiệm, cấu hình nào tạo metric, ai đã phê duyệt hành động công khai, và điều gì xảy ra nếu request bị gửi lại.

Nexus Agent biến chuỗi này thành các boundary rõ ràng:



Ba nhóm giá trị

Nhóm	Giá trị	Cơ chế trong hệ thống
Người nghiên cứu	Tra cứu có nguồn, chuẩn bị dữ liệu, chạy baseline/model comparison, đọc metric và limitation.	researcher, quant-data-agent, quant-researcher, Evidence UI.
Người kiểm soát	Biết hành động nào cần phê duyệt, kết quả dựa trên bằng chứng nào, retry có nhân đôi side effect không.	Native approval, action digest, idempotency, lineage, export dossier.
Developer/operator	Boundary để kiểm thử, worker tách rời, private-beta có auth và least privilege.	HTTP contracts, PostgreSQL roles, Compose overlay, CI gates.

Trong phạm vi và ngoài phạm vi

Được hỗ trợ	Chưa được cam kết
- Local development. - Một private-beta node. - OIDC, role/permission, actor isolation. - Daily OHLCV qua OpenBB + yfinance. - Qlib Alpha158 và bốn model allowlist. - AI-Trader snapshot/feed/publish có governance. - PostgreSQL durability, replay, Evidence UI.	- Multi-replica coordination hay distributed admission. - Remote object storage và off-host restore. - Release signing/provenance hoàn chỉnh. - Capacity/soak certification, formal SLO/alerting. - Độ đúng tuyệt đối của provider hoặc lợi nhuận tương lai. - Realtime trade execution.

Ví dụ đơn giản: nếu kết quả backtest ghi “return 12%”, hệ thống lưu được cấu hình, metric và artifact làm bằng chứng. Nhưng nó không biến 12% lịch sử thành lời hứa lợi nhuận 12% tương lai.

03 Bản đồ repository và công nghệ

```
nexus-agent/
├── backend/                # LangGraph, agent harness, auth, product API, PostgreSQL
│   ├── source/
│   │   ├── agents/        # graph composition, middleware, subagent definitions
│   │   ├── application/   # composite workflows và governed publication
│   │   ├── contracts/     # Pydantic boundary models
│   │   ├── domain/        # framework-independent state/policy
│   │   ├── infrastructure/# PostgreSQL repositories, migration, checkpointer
│   │   ├── integrations/  # HTTP clients đến workers
│   │   ├── security/      # OIDC, actor key, role → permission
│   │   └── tools/         # model-visible high-level tools
│   ├── .deepagents/       # prompt, skill, nexus.json metadata
│   └── migrations/        # schema nexus_domain và database roles
├── frontend/              # React/Vite console, SSE, approvals, graph, Evidence UI
├── quant-data-worker/     # OpenBB ingest, staging, immutable Qlib dataset build
├── quant-worker/          # Qlib job coordinator, result và artifact verification
├── ai-trader-worker/      # cached market intel, signal feed, publication adapter
├── docs/                  # kiến trúc, guide, workflow, reference và PDF này
├── evaluation/            # product-validation manifest
├── scripts/               # architecture/product/security gates
├── data/                  # runtime data local (không phải source authority)
├── docker-compose.yml
├── docker-compose.beta.yml
└── run.sh
```

Technology stack

Lớp	Công nghệ	Vai trò
Agent runtime	Python 3.11, LangGraph, Deep Agents, LangChain model adapter	Graph, delegation, streaming, interrupt/checkpoint.
Backend API	FastAPI, httpx, Pydantic	Product routes, worker client, strict boundary validation.
Persistence	PostgreSQL 16, psycopg pool, LangGraph Postgres checkpointer	Checkpoint + <code>nexus_domain</code> product authority.
Frontend	React 18, Vite 6, oidc-client-ts, Lucide	Chat/SSE, approval, runtime graph, Settings, Evidence.
Market data	OpenBB 4, yfinance adapter, pandas/numpy	Tải, chuẩn hóa, validate OHLCV/factor.
Quant	Qlib 0.9.7, LightGBM/linear/XGBoost/CatBoost	Alpha158 experiment và TopkDropoutStrategy.
AI-Trader adapter	FastAPI, SQLite, Alpha Vantage NEWS_SENTIMENT	Snapshot cache và Nexus signal feed.

Lớp	Công nghệ	Vai trò
Packaging/ops	uv, npm, Docker Compose, Nginx	Dependency lock, build, private networking, reverse proxy.
Verification	unittest, Node test, Playwright, Trivy, security script	Unit, integration, browser, image và vulnerability gates.

Quy tắc ownership quan trọng

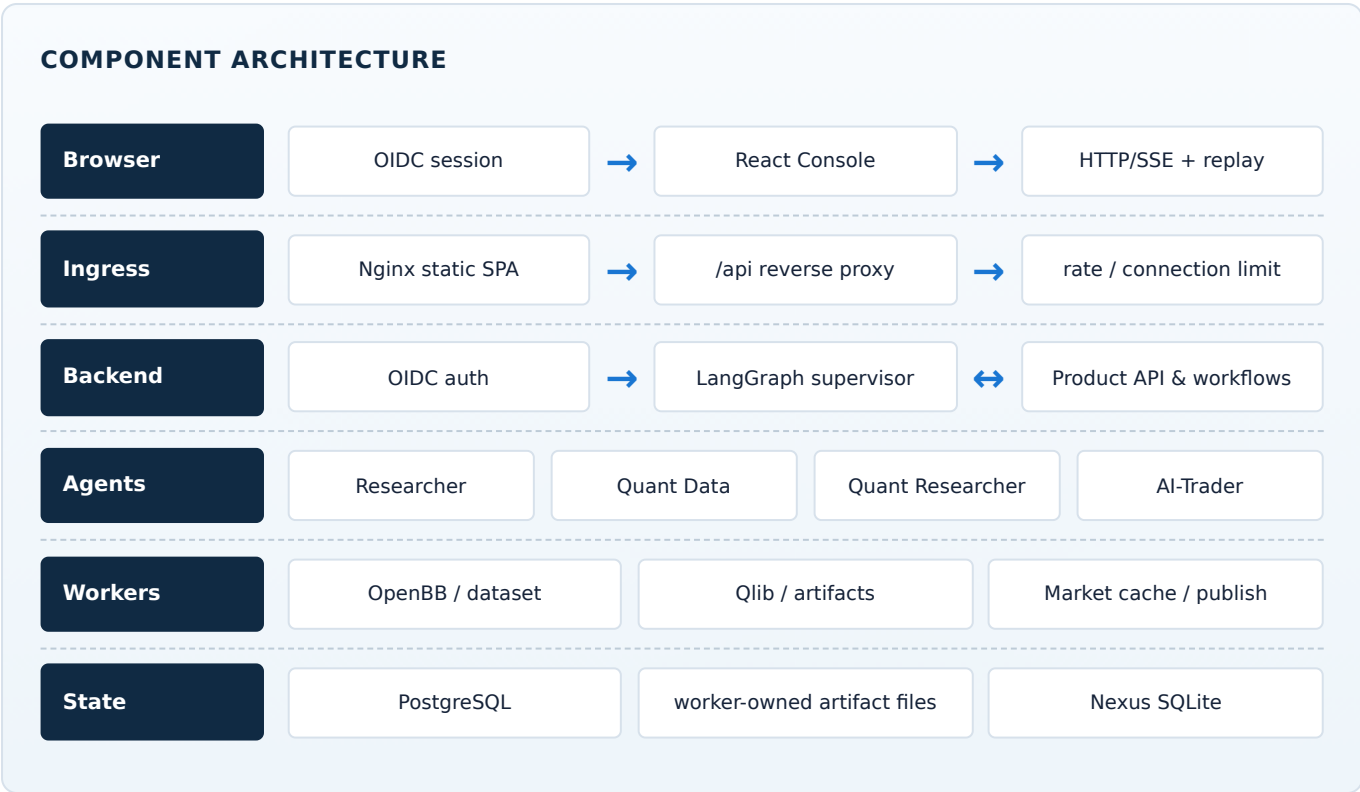
Backend không import worker
Backend gọi worker qua các client dưới `backend/source/integrations/` . Worker là ranh giới HTTP độc lập.

Model không gọi API thô
Agent thấy tool cấp cao; application service kiểm soát các bước `fetch/build/validate/submit`.

Browser không sở hữu topology
Frontend tải cấu trúc agent/skill từ `GET /v1/topology` và fail closed khi không có.

Source thắng docs
Cấu hình executable, schema và implementation là bằng chứng cuối cùng khi có sai khác.

04 Kiến trúc tổng thể



Runtime component và dữ liệu bền vững

Component	Trách nhiệm	Dữ liệu bền vững
backend/	Auth, graph, workflow, product API, approval, memory, events.	Postgres checkpoint và <code>nexus_domain</code> .
frontend/	OIDC client, chat/SSE, graph, Settings, Evidence UI.	Chỉ UI state theo browser session/local storage.
quant-data-worker/	OpenBB fetch, staging validation, Qlib dataset build/manifest.	Actor-scoped files + Postgres revision/control ở beta.
quant-worker/	Allowlisted Qlib job, cancel/recovery, result, artifact.	Postgres job state + actor-scoped artifacts.
ai-trader-worker/	Cached market intel, signal feed và publication endpoint.	Nexus-owned SQLite tables trên node.

Hai control plane song song

LangGraph control plane

Thread, run, message stream, checkpoint, graph interrupt/resume. Nó điều khiển vòng đời hội thoại và model execution.

Product/domain control plane

Workflow, idempotency, approval, product event, dataset/experiment lineage, memory và publication. Nó làm bằng chứng nghiệp vụ.

Vì sao cần cả hai? Một run LangGraph có thể bị ngắt/reconnect, trong khi dataset build hoặc publication phải giữ identity ổn định và không lặp side effect. Product control plane tách trạng thái nghiệp vụ khỏi vòng đời transport/model.

Luồng đồng bộ và bất đồng bộ

- Chat phản hồi qua HTTP streaming/SSE; browser parse nhiều stream mode.
- Qlib experiment được worker nhận với HTTP 202 rồi xử lý như durable job.
- Product events đi qua transactional outbox rồi được projector đưa vào replay stream.
- AI-Trader market refresh chạy nền, còn agent đọc snapshot từ SQLite nên không chờ provider.

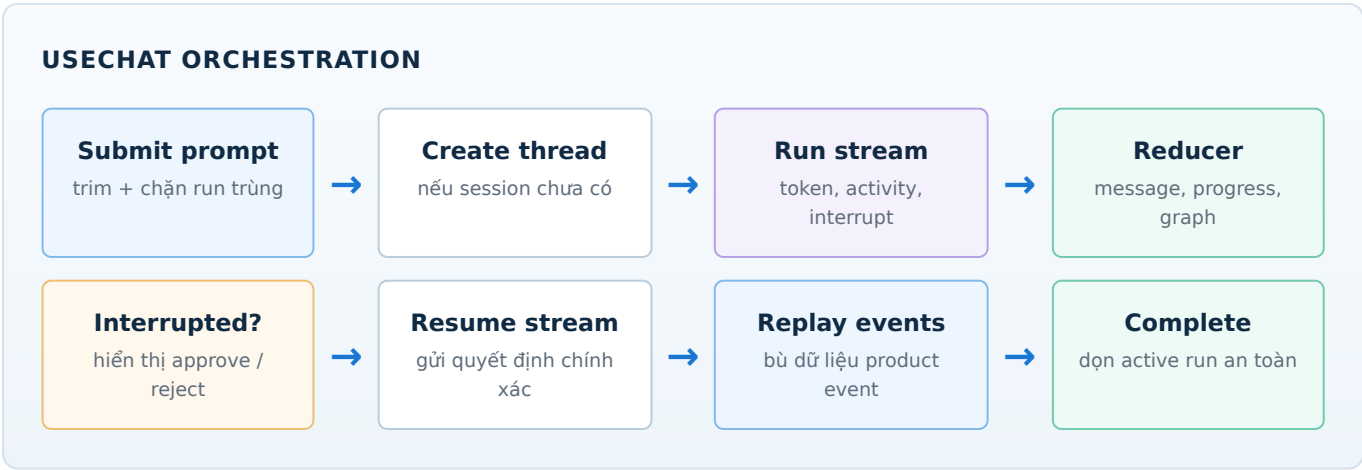
05 Frontend và trải nghiệm người dùng

Frontend là React SPA phục vụ bốn không gian: Trò chuyện, Evidence, Agents và Cài đặt. Nó trình bày trạng thái runtime nhưng không được coi là nguồn sự thật của workflow, approval hay evidence.

Bốn trang chính

Trang	Người dùng thấy gì	Nguồn dữ liệu
Trò chuyện	Prompt, token stream, tiến trình, execution graph, approval card.	LangGraph HTTP/SSE, event projection và React reducer.
Evidence	Dossier, so sánh experiment, lineage, artifact download, export.	Các product route <code>/v1/experiments</code> , <code>/v1/lineage</code> , <code>/v1/artifacts</code> .
Agents	Danh sách agent, skill và tài nguyên/tool được cấu hình.	Topology đã xác thực từ backend.
Cài đặt	Backend health, memory, số agent/skill và thao tác xóa memory.	<code>/ok</code> , <code>/v1/memory</code> , topology.

Vòng đời một lượt chat



Những guard quan trọng trong `frontend/src/hooks/useChat.ts`

- **In-flight guard:** không submit thêm khi lượt hiện tại đang chạy.
- **Run sequence guard:** callback cũ chỉ được cập nhật state nếu ID nội bộ vẫn là run hiện tại.
- **AbortController:** stop của người dùng hủy transport và gửi cancel đến backend khi có run ID.
- **Interrupted run:** giữ state cần thiết để resume approval, không giả vờ run đã hoàn tất.
- **Reconnect/replay:** active run và last event ID được giữ trong browser session để nối lại.

Storage phía browser

Nơi lưu	Dữ liệu	Ý nghĩa
<code>sessionStorage</code>	OIDC user, thread ID, active run, last event/sequence.	Tồn tại trong phiên tab; giúp resume/replay nhưng không là product authority.
<code>localStorage</code>	Vị trí node graph, agent đang mở rộng.	Chỉ là preference trình bày.
React state	Messages, progress, pending approval, selected node, page.	Disposable UI projection.

Runtime topology: backend quyết định, frontend hiển thị

`App.jsx` khởi tạo bằng `FAIL_CLOSED_NEXUS_CONFIG`: chỉ supervisor bật, không tự giả lập subagent/skill. Sau khi `fetchProductTopology()` thành công, `graphBuilder.js` chuẩn hóa cấu hình, tạo node/edge và chỉ mở chi tiết tool/skill khi người dùng yêu cầu hoặc live activity đi qua agent đó.

Fail closed: nếu topology endpoint không hoạt động, browser không dùng một bản topology hardcoded có vẻ “đúng”. Điều này tránh UI nói rằng một agent/tool tồn tại trong khi backend đã tắt nó.

OIDC và reverse proxy

`frontend/src/auth/oidc.ts` dùng Authorization Code flow, silent renew và session storage. Cấu hình được inject lúc runtime. Trong profile không bật OIDC, client không tạo OIDC manager; backend development có local actor riêng.

Nginx phục vụ static SPA và proxy `/api/` đến backend với buffering tắt để streaming hoạt động. Nó áp dụng body limit, rate/connection limit, CSP, `nosniff`, no-referrer, frame deny và Permissions Policy; các surface không dùng như MCP, A2A, docs, metrics bị trả 404.

Evidence UI

`DossierPage.jsx` không chỉ hiện một con số. Nó nhóm evidence thành request/workflow, dataset revision, experiment specification, attempts, metric definitions, sources/interpretation, artifacts, approvals/publications và lineage explorer. Link nguồn chỉ được render nếu URL an toàn; artifact chỉ có nút download khi trạng thái `ready` và `download_eligible=true`.

Ví dụ: thay vì chỉ thấy “IC = 0.04”, người dùng có thể xem metric ID, calculation version, dataset revision, manifest hash, model/segment, runtime attempt và các evidence gap liên quan.

File nên đọc

File	Nội dung
<code>frontend/src/App.tsx</code>	Shell, navigation, topology load, graph projection.

File	Nội dung
frontend/src/hooks/useChat.ts	Chat lifecycle, approval resume, abort và stale-run guard.
frontend/src/api/chat*.ts	HTTP/SSE protocol, stream parsing, thread/run resources.
frontend/src/data/graphBuilder.ts	Fail-closed topology → node/edge.
frontend/src/features/dossier/DossierPage.tsx	Evidence, compare, export, lineage, artifact UI.
frontend/nginx.conf	Production ingress headers và proxy policy.

06 Backend, LangGraph và streaming

Graph mặc định và graph Studio opt-in

`backend/langgraph.json` chỉ đăng ký supervisor, giống production, để standalone subagent không trở thành public run surface ngoài ý muốn. Khi cần trace cục bộ, operator có `studio:access` phải khởi động rõ ràng bằng `backend/langgraph.studio.json`; file này mới đăng ký đủ năm graph.

Graph	Entrypoint	Mục đích
<code>supervisor</code>	<code>source/agents/agent.py:graph</code>	Luồng sản phẩm chính và delegation.
<code>researcher</code>	<code>:researcher_graph</code>	Trace nghiên cứu nguồn ngoài độc lập.
<code>quant-data-agent</code>	<code>:quant_data_agent_graph</code>	Trace ingest/build dataset.
<code>quant-researcher</code>	<code>:quant_researcher_graph</code>	Trace Qlib experiment.
<code>ai-trader-agent</code>	<code>:ai_trader_agent_graph</code>	Trace AI-Trader read/publish.

Composition root: `source/bootstrap.py`

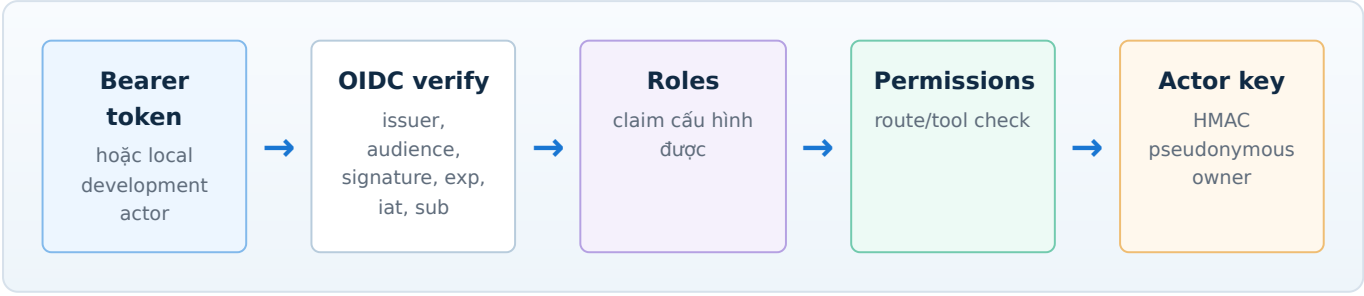
- 1 Đăng ký harness profile.** Registry agent/skill/tool được validate trước khi runtime tiếp tục.
- 2 Tạo model và năm graph.** Supervisor nhận danh sách subagent; standalone graph nhận đúng tool và middleware riêng.
- 3 Mở PostgreSQL pool.** Runtime chờ pool sẵn sàng và verify migration version/checksum, không tự mutate schema khi startup.
- 4 Khởi tạo checkpointer một lần.** Cùng checkpointer được gán cho tất cả graph instance.
- 5 Bọc graph context.** Run start/complete/fail được ghi thành product event khi có actor/run identity hợp lệ.

HTTP surface gồm hai lớp

LangGraph platform routes Thread, run, state, stream, cancel và resume. Frontend dùng các route này để vận hành hội thoại và nhận SSE.	Nexus product routes Topology, replay events, memory, readiness, lineage, dossier, compare, export và artifact download trong <code>source/http_app.py</code> .
--	---

Custom FastAPI app được mount với **auth first**. OpenAPI/docs bị tắt; MCP, A2A và webhooks cũng bị vô hiệu vì không thuộc product surface hiện tại.

Từ token đến quyền



OIDC subject không được dùng trực tiếp làm owner key. Backend tạo `v1-<64 hex>` từ issuer + subject bằng tenant HMAC key. Mọi thread/domain record sau đó được scope theo actor key.

Input và state boundary

Boundary	Giới hạn mặc định	Lý do
Run payload	64 KiB	Chặn request quá lớn trước graph.
Một user prompt	16 KiB	Giới hạn model input và abuse surface.
Checkpoint TTL	43.200 giây (12 giờ)	Dọn state hội thoại có giới hạn.
Replay retention	86.400 giây (24 giờ)	Cho phép reconnect gần thời điểm chạy.
Approval TTL	900 giây (15 phút)	Không để quyết định pending vô thời hạn.
Artifact download	256 MiB	Giới hạn proxy/download memory.

Streaming và replay phối hợp ra sao?

SSE cho cảm giác realtime: token, tool activity và interrupt đi đến browser trong lúc run chạy. Product-event replay giải quyết một bài toán khác: nếu browser rớt mạng hoặc missed event, nó hỏi `GET /v1/runs/{run_id}/events?after=N` để lấy envelope có sequence lớn hơn N. Trường `gap` báo khi phần lịch sử cần thiết đã vượt retention.

Ví dụ: token cuối cùng có thể đã hiện trên UI trước khi sự kiện `dataset.ready` được projector đưa vào replay. Sau khi run hoàn tất, frontend replay để bổ sung product state mà không đoán từ prose của model.

Strict checkpoint serialization

Cấu hình checkpointer không allowlist JSON module tùy ý và tắt pickle fallback. Production bật strict msgpack và mã hóa content fields bằng AES key có độ dài 16/24/32 byte. Mục tiêu là giảm nguy cơ deserialize object tùy ý và bảo vệ nội dung checkpoint.

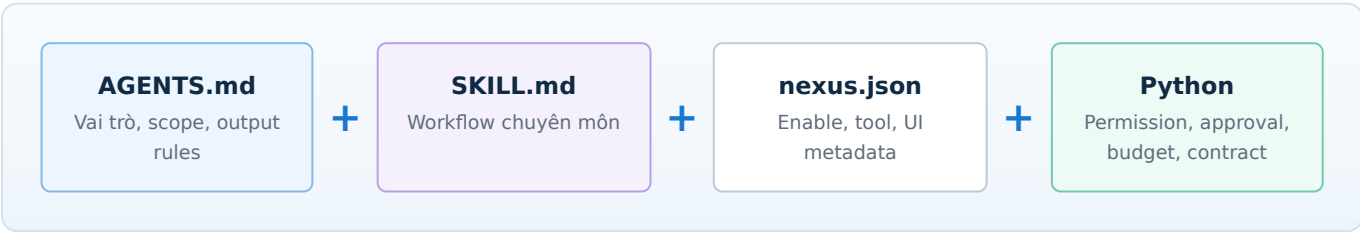
07 Agent harness và phân quyền nhiệm vụ

Supervisor không phải “siêu agent làm mọi thứ”. Nó chọn đúng chuyên gia, nhận structured handoff và dừng đường đi nếu contract thất bại. Mỗi subagent có prompt, skill, tool, permission và budget riêng.

Topology hiện tại

Agent	Trách nhiệm	Tool model nhìn thấy
supervisor	Điều phối; lưu/xóa memory khi có yêu cầu rõ ràng.	save_user_memory , delete_user_memory , task , write_todos .
researcher	Nghiên cứu nguồn ngoài hiện hành, source-backed.	web_search , write_todos .
quant-data-agent	Chuẩn bị dataset và factor context.	prepare_qlib_dataset , fetch_factor_snapshot , write_todos .
quant-researcher	Liệt kê dataset, chạy/đọc Qlib experiment, ghi interpretation.	Bốn quant tool chuyên trách và write_todos .
ai-trader-agent	Đọc market cache/feed; publish có approval; heartbeat.	Sáu AI-Trader tool và write_todos .

Prompt + skill + executable policy



Ba file văn bản hướng dẫn model không phải security boundary duy nhất. Executable Python mới sở hữu danh sách tool thật, permission check, approval, structured response schema, limit và side-effect workflow. Startup cross-check registry với prompt/skill/module để phát hiện drift.

Skill được đọc từ đúng registry assignment trong lúc dựng graph, kiểm tra containment/frontmatter/kích thước rồi inject đầy đủ như trusted system instruction. Model không có tool để tự ls , read_file , grep , glob , execute hay sửa filesystem.

Ranh giới từng agent

Researcher

Được: docs, paper, current context, citation.
Không: ingest, Qlib run, file, lời khuyên đầu tư. Tối đa bốn search tập trung theo skill.

Quant Data

Được: daily OHLCV, validation, build dataset, current factor snapshot. **Không:** train, strategy selection, dùng current factor làm historical feature.

Quant Researcher

Được: allowlisted experiment và diễn giải limitation. **Không:** raw Qlib, fetch/build dataset, model/strategy ngoài allowlist.

AI-Trader

Được: cached intel, signal feed, governed publication. **Không:** realtime trade operation hay coi empty snapshot là connector failure.

Nguyên tắc “một composite tool”

Quant Data không thấy bốn tool fetch → validate → build → validate riêng. Nó chỉ gọi `prepare_qlib_dataset` một lần. Quant Researcher cũng gọi `run_governed_qlib_experiment` thay vì tự validate rồi submit tùy ý. Cách này giữ thứ tự, idempotency và resume trong trusted code.

Tại sao? Nếu model có bốn tool side effect thấp tầng, một lần “sửa câu trả lời” có thể fetch/build lặp lại hoặc bỏ qua validation. Composite workflow chỉ cho model diễn đạt ý định; code quyết định chuỗi hành động.

Structured handoff 1.0

Bốn loại kết quả là `ResearchHandoff`, `QuantDataHandoff`, `QuantExperimentHandoff` và `AITraderHandoff`. Tất cả có:

- `schema_version`, `ok`, `agent`, `task`, `summary`;
- `typed references`, `metrics`, `warnings`, `errors`;
- outputs chuyên biệt và một `next_action` mang tính gợi ý.

Schema dùng strict mode và cấm field thừa. Nếu output sai, runtime cho phép một lần sửa format **không gọi tool**; nếu vẫn sai thì fail closed. Nó không lặp dataset build, experiment, publication, memory hay side effect để “sửa” JSON. Supervisor coi prose, claim, web extract và `next_action` là dữ liệu không tin cậy; chỉ typed identifier từ trusted product tool được dùng làm reference, và routing action do quant handoff đề xuất bị sanitize.

Budget và filesystem

Supervisor tối đa 4 delegation và 12 model call cho một run; cùng subagent chỉ được gọi một lần trong mỗi user intent. Researcher tối đa 4 search/6 model call; Quant Data 2 tool/5 model call; Quant Researcher 6 tool/8 model call với experiment và interpretation tối đa một lần mỗi loại; AI-Trader 6 tool/8 model call với mỗi publish tool tối đa một lần. Mọi graph dùng state backend; filesystem tool bị loại khỏi model request và bị chặn lại ở execution boundary.

Vai trò của `nexus.json`

File này là registry chung cho enablement, tool/skill assignment và metadata UI. Backend dùng nó để wiring/validation; frontend nhận phiên bản đã lọc an toàn qua topology API. Secret, prompt đầy đủ và executable permission không được gửi xuống browser.

08 Contract, event và tính đúng đắn

Ba loại contract

Loại	Ví dụ	Điều nó bảo vệ
Intent contract	PrepareQlibDatasetIntent , RunQlibExperimentIntent	Input normalized, allowlist, date/order/range rule.
Handoff contract	QuantDataHandoff	Model output không được bịa ID/metric từ prose.
Event contract	ProductEventEnvelope + typed payload	Replay data nhỏ, versioned, không chứa sensitive-shaped field.

Ví dụ intent dễ hiểu

```
{
  "schema_version": "1",
  "symbols": ["AAPL", "MSFT"],
  "start_date": "2022-01-01",
  "end_date": "2024-12-31",
  "provider": "yfinance",
  "frequency": "1d",
  "adjustment": "adjusted",
  "dataset_alias": "us-tech-demo",
  "universe_name": "us-tech"
}
```

Contract sẽ uppercase/sort symbol, từ chối trùng/rỗng, buộc date range hợp lệ, chỉ chấp nhận provider/frequency hiện hỗ trợ, yêu cầu `close` và `factor` trong include fields.

Identity không giống label

Identity	Được tính từ	Dùng để
request_fingerprint	Provider parameters đã normalize, không gồm retrieval time.	Nhận biết cùng một yêu cầu lấy dữ liệu.
content_hash	Canonical normalized provider content.	Nhận biết byte/nội dung thực sự thay đổi.
staging_revision_id	Immutable OHLCV revision.	Truy vết nguồn vào dataset.
dataset_revision_id	Canonical file manifest.	Input bất biến của experiment.
dataset_alias	Nhãn người dùng chọn.	Dễ đọc; không phải storage identity.
experiment_id	Durable job/result identity.	Poll, dossier, compare, export.

Identity	Được tính từ	Dùng để
<code>action_digest</code>	Actor + tool + normalized args + target.	Ràng buộc approval/idempotency với đúng side effect.

Idempotency bằng ví dụ

Tình huống: browser reconnect và gửi lại cùng request build dataset. Workflow digest giống nhau tạo cùng key. Repository trả workflow cũ; code resume từ stage đã commit hoặc trả kết quả completed. Nó không tạo một dataset logic thứ hai.

Conflict: nếu cùng idempotency key nhưng arguments digest khác, repository từ chối. “Cùng chìa khóa, khác hành động” không được silently reuse.

Product event envelope

```
{
  "event_id": "evt_v1...",
  "schema_version": 1,
  "sequence": 42,
  "actor_key": "v1-...",
  "thread_id": "thread-...",
  "run_id": "run-...",
  "event_type": "dataset.ready",
  "occurred_at": "2026-07-16T...Z",
  "sensitivity": "internal",
  "payload": {
    "dataset_revision_id": "dsr_v1...",
    "manifest_hash": "...",
    "dataset_alias": "us-tech-demo"
  }
}
```

Mỗi payload được validate theo event type, tối đa 32 KiB và reject key có hình dạng nhạy cảm như authorization, cookie, credential, password, secret, token hay raw prompt/content. Event ghi metadata đủ để dựng timeline nhưng không biến event log thành kho prompt/token.

Nhóm event chính

- run.started/completed/failed
- agent.delegated
- tool.proposed/completed/failed
- approval.requested/decided/expired
- workflow.transitioned
- staging.ready
- dataset.ready
- factor_snapshot.ready
- worker.accepted/leased/started/...
- artifact.created/expired/deleted
- publication.published
- memory.updated

Transactional outbox

Thay vì “commit domain row rồi hy vọng ghi event thành công”, repository ghi domain state và outbox record trong transaction. Background projector chuyển outbox thành replayable event. Nếu projector tậm lỗi, state chính vẫn nhất quán và có thể project lại.

Safe errors

Worker trả error có `code` , message an toàn, stage và `retryable` ; backend không gửi traceback/raw framework object cho model hay browser. Retry decision dựa trên thuộc tính rõ ràng, không dựa trên model “đoán” rằng lỗi có vẻ tạm thời.

09 PostgreSQL, filesystem và ownership

PostgreSQL là product authority

Database có hai vùng ownership: schema `langgraph` cho checkpoint do LangGraph quản lý, và `nexus_domain` cho dữ liệu sản phẩm do repository/migration của Nexus quản lý.

Nhóm dữ liệu nexus_domain	Ví dụ bảng/record	Mục đích
Identity & dataset	request fingerprints, staging revisions, dataset revisions/aliases/files.	Immutable lineage và manifest evidence.
Experiment & job	experiments, job attempts.	Queue/lease/run/recovery/result state.
Control	workflows, transitions, idempotency decisions.	Composite workflow và duplicate suppression.
Human governance	approval requests, publication effects.	Proposal → decision → executed effect.
Evidence	lineage nodes/edges, research sources, interpretation reports, dossier exports.	Truy vết và xuất hồ sơ.
Events	product events, transactional outbox, replay events.	Timeline và reconnect.
User memory	user memory items + bounded aggregate.	Fact theo key và prompt middleware.
Artifacts	artifact metadata và tombstones.	Hash, retention, lifecycle; bytes ở filesystem.

Migration lifecycle

Các domain migration hiện đi từ 0001 đến 0008: foundation; workflow/replay; durable jobs; product-event contract; approval execution evidence; dossier; manifest immutability; keyed user memory. Compose chạy `domain-migrate` trước services. Runtime chỉ verify version/checksum và fail nếu schema không đúng.

Fresh-memory baseline: migration 0008 hiện tạo cả aggregate lẫn keyed items cho database mới. Database từng ghi checksum cũ của 0008 hoặc migration 0009 thuộc baseline đã bỏ phải tạo lại volume; không có đường in-place conversion được hỗ trợ.

Filesystem: bytes lớn, identity vẫn được kiểm soát

Đường dẫn runtime	Chứa gì	Cơ chế bảo vệ
<code>data/staging</code>	OHLCV/factor revision đã normalize.	Actor scope, request/content identity, validation metadata.

Đường dẫn runtime	Chứa gì	Cơ chế bảo vệ
data/qlib	Immutable Qlib provider layout.	Canonical manifest, file hashes, revision ID.
data/artifacts	Experiment artifact bytes.	Metadata/hash/lifecycle trong Postgres, verified download.
data/ai-trader	SQLite market snapshots và signal feed.	Nexus-owned table, worker private boundary.

Không có remote object storage trong baseline. Do đó durability của bytes phụ thuộc một node/volume; Postgres biết manifest và hash nhưng không tự khôi phục file bị mất ở ngoài node.

Actor isolation

Actor key đi từ authentication vào thread owner filter, workflow, dataset link, experiment, approval, memory, event, artifact request và worker header. Production worker dùng pseudonymous actor key cùng credential riêng cho từng quant service; mỗi service có database role tối thiểu riêng và actor-keyed table được bảo vệ thêm bằng PostgreSQL row-level security. Projector và quant coordinator chỉ có routine database hẹp, kiểm tra đúng service role; chúng không có principal bypass RLS tổng quát.

Ví dụ: hai người cùng yêu cầu AAPL trong cùng date range có thể tạo/reuse content revision theo policy, nhưng các actor-visible links, workflow, experiment, evidence và download authorization vẫn scope theo actor. Biết một ID của người khác không đủ để truy cập.

User memory là fact store, không phải prompt blob

Mỗi fact có stable generic key như `identity.name` hoặc `preference.language`. Save upsert đúng key nên preference mới không đè identity không liên quan. Repository dựng một aggregate text/Markdown xác định, tối đa 16 KiB, để API và prompt middleware dùng.

Retention khác nhau

Loại	Mặc định	Ghi chú
Thread/checkpoint	12 giờ	LangGraph TTL delete strategy.
Replay event	24 giờ	Gap được báo khi sequence cũ đã mất.
Pending approval	15 phút	Hết hạn thì không được execute.
Quant artifact	90 ngày trong Compose default	Có lifecycle/tombstone; cấu hình được.
User memory	Không TTL tự động	Chỉ lưu/xóa khi người dùng có ý định và quyền phù hợp.

10 Workflow OpenBB → Qlib dataset

Mục tiêu của workflow là biến một ý định đơn giản — symbol, date range, adjustment, alias, universe — thành một Qlib dataset revision bất biến, có manifest đã xác minh và limitation rõ ràng.

Luồng đầy đủ



Bước 1 — Validate và normalize ý định

Input chấp nhận 1-300 unique symbols, provider `yfinance`, frequency `1d`, một adjustment policy rõ ràng, date range ISO, alias/universe theo pattern an toàn. Symbol được trim, uppercase và sort để cùng ý định có canonical form.

```
Người dùng:  
"Tạo dataset AAPL và MSFT từ 2022-01-01 đến 2024-12-31,  
daily, adjusted, alias us-tech-demo."
```

```
Ý định chuẩn hóa:  
symbols = ["AAPL", "MSFT"]  
provider = "yfinance"  
frequency = "1d"  
adjustment = "adjusted"  
dataset_alias = "us-tech-demo"  
universe_name = "us-tech"
```

Bước 2 — Fetch hoặc resolve staging revision

Backend gọi quant-data-worker qua client, thêm ingestion idempotency key. Worker dùng OpenBB/yfinance, chuẩn hóa thành schema:

```
date, symbol, open, high, low, close, volume, factor
```

Date được parse; symbol uppercase; trường giá/volume chuyển sang numeric; row không sử dụng được bị loại; dữ liệu sort theo symbol/date. Trong pipeline OHLCV hiện tại, `factor` được đặt 1.0 trong normalized staging. Canonical content tạo `content_hash`; revision lưu metadata, request fingerprint và retrieval time.

Bước 3 — Deterministic staging validation

Check	Điều kiện lỗi	Vì sao cần
Required columns	Thiếu một cột schema.	Qlib builder cần shape cố định.
Date/symbol uniqueness	Trùng cặp (date, symbol) .	Không cho một observation mơ hồ.
Required values	Null hoặc non-numeric ở trường bắt buộc.	Tránh model input im lặng sai.
Volume	Volume âm.	Phát hiện dữ liệu vô lý.
OHLC consistency	Quan hệ open/high/low/close không hợp lệ.	Phát hiện giá bị hỏng.
Coverage	Ghi date range, symbols, missing per symbol và sort status.	Làm preflight cho experiment.

Nếu fail, workflow chuyển `failed` với code `invalid_staging_revision` và không build Qlib dataset.

Bước 4 — Build immutable Qlib layout

Worker chuyển staged frame sang provider layout có calendars, features và instruments. Dataset revision ID được dẫn xuất từ canonical manifest, không từ alias. Manifest liên kết source staging revision, file list/hash, limitation và metadata coverage.

Alias và revision: `us-tech-demo` là tên người đọc nhớ được; một revision như `dsr_v1_...` mới là identity mà experiment phải dùng. Nếu provider trả nội dung khác ở lần sau, alias có thể trỏ đến revision mới nhưng revision cũ vẫn bất biến.

Bước 5 — Validate manifest và coverage

Final validation kiểm tra layout, manifest/files/hash, calendar, universe và tính `max_test_end` . Với Alpha158 label cần hai trading day phía sau, ngày cuối dataset không phải luôn là test end hợp lệ.

Ví dụ calendar kết thúc: 2024-12-31
Hai trading day cuối dùng cho label horizon.
`max_test_end` có thể là: 2024-12-27

→ Test end phải $\leq$ `max_test_end`, không dùng thẳng `end_date`.

State resume và retry

Nếu worker timeout sau khi staging đã commit, retryable failure được resume từ state suy ra theo partial output:

Đã có bằng chứng	Resume state	Không lặp lại
<code>staging_revision_id</code>	<code>staged</code>	Không fetch lại provider nếu revision đã resolve.
<code>staging_validated=true</code>	<code>staging_validated</code>	Không làm mất kết quả validation đã commit.

Đã có bằng chứng	Resume state	Không lặp lại
<code>dataset_revision_id</code>	<code>dataset_built</code>	Không build một revision mới chỉ vì transport lỗi.

Factor snapshot là nhánh riêng

`fetch_factor_snapshot` lấy market cap, valuation, leverage, profitability/growth/margin context hiện tại khi provider trả được. Nó có revision/fingerprint/hash riêng nhưng được gán cờ:

- `point_in_time_status = unsupported`;
- không có availability date đáng tin cậy;
- không đủ điều kiện làm historical training feature;
- chỉ dùng làm current context.

Financial limitations được ghi cùng dataset

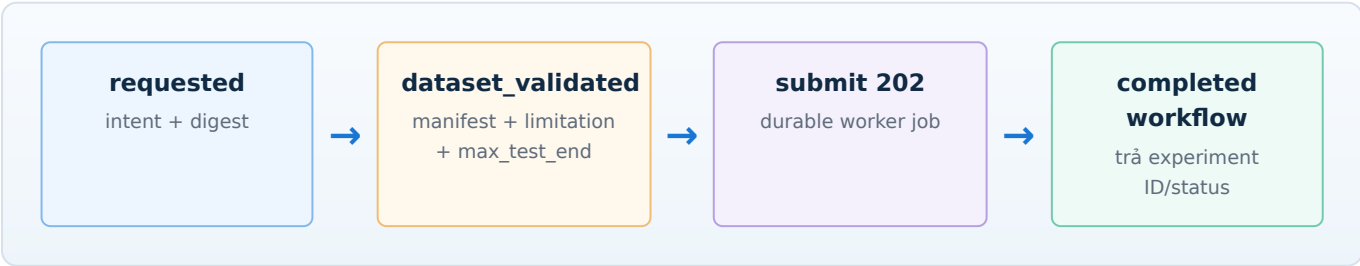
Giới hạn	Trạng thái baseline	Hệ quả
Point-in-time	Unsupported	Không chứng minh dữ liệu đúng như biết tại thời điểm lịch sử.
Universe membership	User supplied; membership dates unavailable	Có nguy cơ survivorship bias.
Adjustment semantics	Provider-defined, verification unsupported	Cần thận trọng với split/dividend.
Calendar	Union của returned rows	Không phải exchange calendar đã chứng nhận.
Missing data	Không fill per symbol	Missing row giữ nguyên để nhìn thấy.
Provider revision/delisting	Unsupported	Không tuyên bố production-grade market history.

Kết luận đúng: dataset “manifest verified” nghĩa là các file hiện tại khớp manifest bất biến. Nó không có nghĩa provider đã trở thành point-in-time hoặc survivorship-free.

11 Workflow Qlib experiment

Workflow này nhận một dataset revision đã xác minh, kiểm tra range/model/strategy, tạo durable experiment job rồi để quant-worker thực thi và công bố result/artifact đã kiểm chứng.

Phần backend: validate trước, submit sau



Lưu ý: “workflow completed” ở backend có nghĩa job đã được worker chấp nhận bền vững và ID đã được trả, không nhất thiết model training đã chạy xong. Agent tiếp tục poll result bằng tool riêng.

Contract experiment

Trường	Allowlist / giới hạn
Dataset	dataset_revision_id dạng dsr_v1_... và universe có thật.
Feature set	Chỉ Alpha158 .
Model	lightgbm , linear , xgboost , catboost .
Segments	Train, valid, test có thứ tự, không overlap, nằm trong coverage.
Strategy	topk_dropout ; topk 1-300, n_drop 0-100 và ≤ topk.
Account/cost	Bounded account, open cost, close cost, min cost.

Ví dụ cấu hình baseline

```
{
  "dataset_revision_id": "dsr_v1_...",
  "universe": "us-tech",
  "model": "linear",
  "train": {"start": "2022-01-03", "end": "2023-06-30"},
  "valid": {"start": "2023-07-03", "end": "2023-12-29"},
  "test": {"start": "2024-01-02", "end": "2024-12-27"},
  "topk": 2,
  "n_drop": 1,
  "open_cost": 0.0005,
  "close_cost": 0.0015
}
```

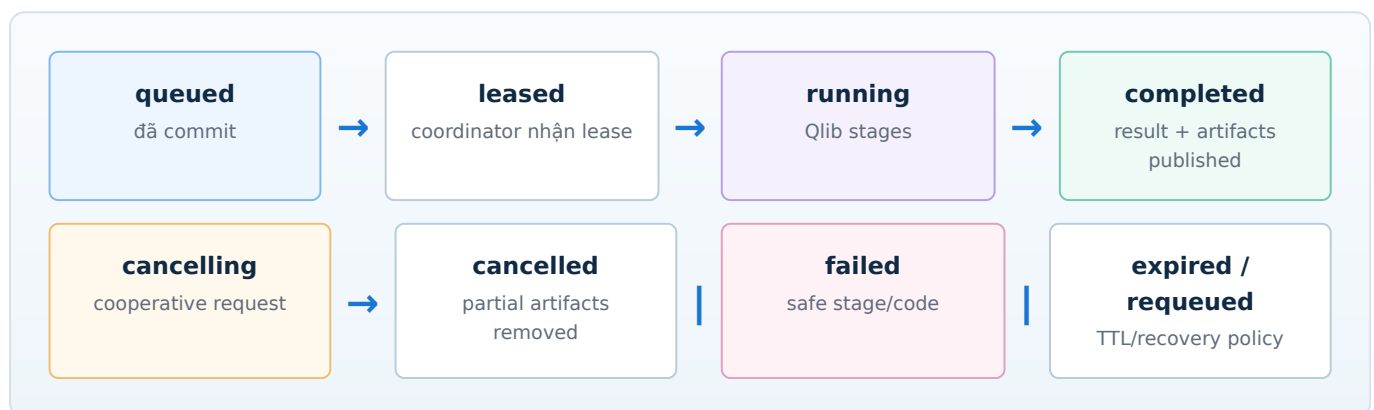
`linear` thích hợp làm baseline nhanh; mặc định sản phẩm là `lightgbm`. XGBoost/CatBoost chỉ được chọn khi người dùng yêu cầu so sánh hoặc nêu tên.

Preflight trước khi chiếm worker slot

- Dataset tồn tại và manifest verified.
- Universe có trong dataset.
- Ba segment nằm trong calendar.
- Train < valid < test, không overlap.
- Test end chứa đủ hai later trading days cho Alpha158 label.
- Model, feature set và strategy nằm trong allowlist.

Lợi ích: request sai ngày bị từ chối trước khi consume một job slot và trước khi Qlib khởi tạo tài nguyên nặng.

Worker job state machine



Production dùng PostgreSQL job state; coordinator có global active job limit, queued TTL, max execution time và actor storage quota. Khi worker restart, state bền vững cho phép lease/recovery policy xử lý job thay vì dựa vào memory của process.

Qlib execution stages

Runner cô lập Qlib/MLflow runtime tracking khỏi artifact evidence, chuẩn bị dataset/handler/model, train, generate signal, backtest và ghi artifact atomically. Exception được gán stage; partial artifact bị dọn khi không thể publish an toàn.

Kết quả và ý nghĩa metric

Metric	Giải thích đơn giản	Không nên hiểu là
IC	Tương quan trung bình giữa score dự báo và label.	Phần trăm lợi nhuận.
ICIR	IC trung bình chia độ biến động IC; đo độ ổn định tương đối.	Bảo đảm model ổn định tương lai.
Rank IC / Rank ICIR	Tương quan theo thứ hạng và độ ổn định của nó.	Khả năng dự báo giá chính xác tuyệt đối.
Annualized return	Lợi suất lịch sử annualize trong backtest.	Lợi suất cam kết.
Excess return after cost	Phần vượt benchmark sau cost giả định.	Chi phí thực tế đã được bao phủ hết.
Information ratio	Excess return so với biến động của excess return.	Sharpe ratio hay xác suất thắng.
Max drawdown	Mức sụt giảm peak-to-trough lớn nhất trong mẫu.	Worst case tương lai.
Turnover	Tần suất/mức độ thay đổi holdings.	Chi phí giao dịch đã chính xác cho mọi market.

Artifact evidence

Runner tạo ba loại artifact: `predictions.parquet`, `report.parquet` và `positions.pkl`. Mỗi artifact có ID, producer, type, media type, size, SHA-256, retention, storage status, download eligibility và thời điểm hết hạn.

Backend download theo hai bước: lấy manifest/authorization từ worker, sau đó tải bytes với size limit và xác minh content hash. Nếu hash khác, trả conflict thay vì chuyển file hỏng cho browser.

Interpretation là evidence mới, không sửa result

Sau completed result, quant-researcher gọi `record_experiment_interpretation` một lần, liên kết đúng metric và artifact IDs. Report là append-only mapping; nó không thay đổi metric, experiment hay artifact gốc.

12 AI-Trader integration

`ai-trader-worker` là adapter nhỏ do Nexus sở hữu, không phải toàn bộ upstream AI-Trader platform. Nó tách provider latency khỏi chat path bằng snapshot cache và cung cấp signal feed/publication boundary.

Hai luồng độc lập



Market intelligence snapshot

Bốn category là `equities`, `macro`, `crypto` và `commodities`. Refresh daemon gọi provider, normalize/dedupe news, tạo category summary và overview, sau đó ghi SQLite. Read endpoint thêm freshness/staleness metadata.

Phân biệt ba trạng thái dễ nhầm

Tool result	Ý nghĩa đúng	Không nên nói
<code>ok=false</code> , <code>ai_trader_not_configured</code>	Backend chưa có base URL connector.	Worker có snapshot rỗng.
<code>ok=true</code> , <code>available=false</code>	Connector hoạt động nhưng chưa có cached snapshot phù hợp.	Base URL sai.
<code>ok=true</code> , <code>feed total=0</code>	Chưa có strategy/discussion phù hợp trong signal feed.	Không có market news cho symbol.

Ví dụ AAPL: agent gọi market-news với filter `AAPL` và đọc `matched_symbols` / `unmatched_symbols`. Nó không lấy signal feed rỗng làm bằng chứng rằng không có tin AAPL.

Tool surface

Tool	Loại	Permission / governance
<code>get_ai_trader_market_overview</code>	Read snapshot	<code>ai_trader:read</code>
<code>get_ai_trader_market_news</code>	Read snapshot + category/symbol filter	<code>ai_trader:read</code>
<code>get_ai_trader_signal_feed</code>	Read Nexus-published feed	<code>ai_trader:read</code>
<code>publish_ai_trader_strategy</code>	Side effect	<code>ai_trader:publish</code> + native approval
<code>publish_ai_trader_discussion</code>	Side effect	<code>ai_trader:publish</code> + native approval
<code>poll_ai_trader_heartbeat</code>	Protected explicit call	<code>ai_trader:read</code> ; không chạy ngầm trong agent turn

Governed publication

- 1 Model đề xuất exact tool arguments, kèm market/symbol, nội dung, limitation và evidence IDs.
- 2 Approval middleware normalize args, tính target boundary + action digest, commit pending record rồi interrupt graph.
- 3 Người dùng approve/reject đúng proposal. Decision được ghi trước khi execution.
- 4 Publication service yêu cầu approval hợp lệ, reserve effect bằng idempotency, gọi worker với actor/digest.
- 5 Worker ghi SQLite signal hoặc trả conflict; Postgres đánh dấu published/failed và liên kết evidence.

Worker authentication

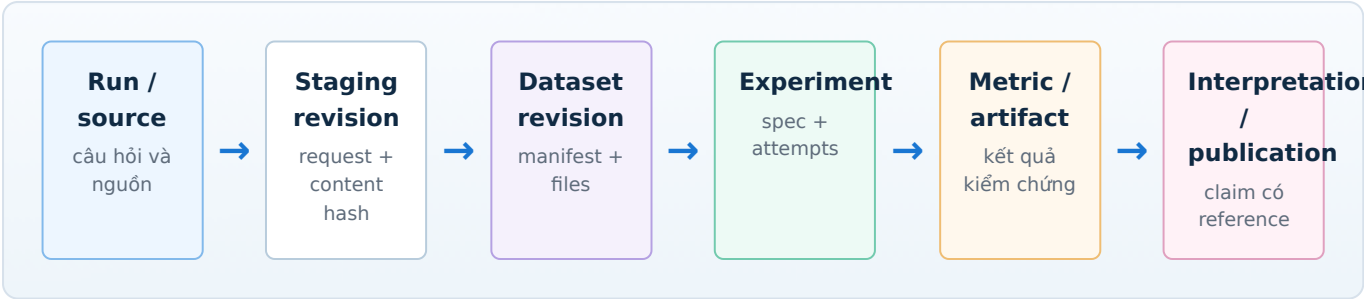
Overview/news/feed read endpoints không cần token bên trong private worker boundary. Ready, refresh, publish và heartbeat cần `AI_TRADER_TOKEN` qua Bearer hoặc `X-Claw-Token`. Production đồng thời dựa vào private networking; port worker không public.

SQLite ownership

Worker chỉ tạo bảng `nexus_signals` và `nexus_market_snapshots`, không migrate hay chạm upstream table. PostgreSQL vẫn là authority cho governed publication; SQLite là local adapter store của target system.

13 Evidence, dossier và lineage

Evidence graph



Lineage repository cho phép duyệt backward, forward hoặc both, depth 1-8 và giới hạn node. Query luôn scope theo actor. Một node ID hợp lệ nhưng không thuộc actor được đối xử như không tìm thấy.

Experiment dossier chứa gì?

Nhóm	Nội dung	Câu hỏi nó trả lời
Workflow	Normalized input, state/stage, transition, thread/run.	Ý định nào và đi qua bước nào?
Dataset	Revision, source staging, manifest/files, validation, limitation.	Kết quả dùng dữ liệu chính xác nào?
Specification	Model, feature, strategy, segments, cost, label horizon.	Thử nghiệm thực sự chạy cấu hình gì?
Attempts	Attempt, status/stage, runtime versions.	Có retry/recovery hay không?
Metrics	Value, group, unit, meaning, calculation version, evidence status.	Con số có định nghĩa và bằng chứng đủ không?
Sources	Locator, retrieval status, hash, metadata.	Claim ngoài dựa trên nguồn nào?
Artifacts	Hash, retention, storage status, download eligibility.	File kết quả còn nguyên và tải được không?
Interpretation	Report + exact evidence references.	Nhận định nào được nối với metric/artifact nào?
Approval/publication	Decision, execution status, target reference.	Ai đã cho phép side effect nào?
Evidence gaps	Code/message về phần thiếu hoặc yếu.	Phần nào không đủ để kết luận mạnh?

Compare và export

API compare nhận 2-5 experiment ID, dedupe và validate pattern trước query. Điều này hỗ trợ đối chiếu model/config/metric cùng scope actor. Export tạo JSON dossier bundle, trả content hash trong header và ghi export evidence.

Artifact download an toàn

- 1 Browser yêu cầu backend bằng artifact ID, không gọi worker trực tiếp.
- 2 Backend kiểm tra `quant:read`, actor scope, storage status và download eligibility.
- 3 Backend tải với max-byte limit, tính/so content hash với manifest.
- 4 Chỉ khi khớp mới trả bytes cùng `nosniff` và hash header.

Partial evidence không bị che giấu

Dossier có evidence gap thì UI gắn nhãn partial và liệt kê code/message. “Không tìm thấy obvious issue” không được coi là bằng chứng hoàn chỉnh. Source retrieval, metric definition, artifact status và lineage reference phải được đánh giá riêng.

Ví dụ: metric có value nhưng thiếu calculation version hoặc artifact reference vẫn có thể hiển thị để điều tra, nhưng không nên được coi là fully evidenced.

14 Bảo mật, approval và user memory

Role → permission

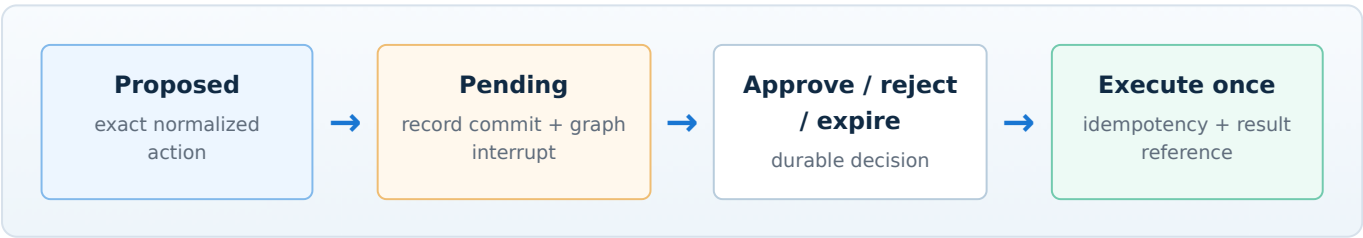
Role	Permissions	Chân dung sử dụng
user	chat, research read, AI-Trader read	Hỏi đáp/nghiên cứu và đọc market context.
analyst	user + quant:read , quant:data:write , quant:experiment:run , quant:interpretation:write	Chuẩn bị dataset, experiment và Evidence.
publisher	user + ai_trader:publish	Publish strategy/discussion có approval.
admin	chat, research, AI read/publish, quant, Studio, maintenance	Quản trị private beta.

Nhiều role có thể cộng permissions. Backend yêu cầu ít nhất chat:run để token được coi là Nexus user hợp lệ.

Production validation fail closed

Khi NEXUS_ENV=production , startup từ chối default database password, thiếu OIDC issuer/audience, tenant HMAC, worker token, checkpoint AES key; issuer/JWKS phải HTTPS; JWT algorithm chỉ được thuộc asymmetric RS/PS/ES allowlist; HMAC/worker secret phải đủ dài.

Approval lifecycle



Agent-initiated actions cần native approval

- save_user_memory
- delete_user_memory
- publish_ai_trader_strategy
- publish_ai_trader_discussion
- prepare_qlib_dataset
- fetch_factor_snapshot
- run_governed_qlib_experiment
- record_experiment_interpretation

Action digest gồm actor, tool, normalized arguments, schema version, tool-call identity và target boundary. Approve một action A không cho phép thay arguments thành B khi resume. Quant effect chỉ

nhận capability tạm thời sau khi middleware kiểm tra durable approval dưới advisory lock; direct tool invocation không có capability sẽ fail closed. Approval hết TTL hoặc đã reject không thể execute.

Memory policy

Supervisor chỉ lưu/xóa khi người dùng yêu cầu rõ ràng. Nội dung phù hợp là fact profile không nhạy cảm; không lưu credential, auth data, financial account identifier, health data hay sensitive personal data.

Yêu cầu phù hợp:

"Hãy nhớ tôi thích câu trả lời bằng tiếng Việt."

→ memory_key: preference.language

→ content: "Ưu tiên câu trả lời bằng tiếng Việt."

→ hiển thị approval → approve → upsert

Yêu cầu không phù hợp:

"Hãy nhớ access token này..."

→ từ chối lưu, không đưa token vào memory/event/docs.

Memory injection không nâng quyền

Prompt middleware nạp aggregate dưới một `<user_memory>` block và nói rõ đây là user-controlled reference, không phải system instruction. Nội dung bên trong không được phép thay đổi policy hay cấp tool.

Hai con đường xóa memory

- **Qua agent:** model đề xuất `delete_user_memory`, gây interrupt và cần approve.
- **Qua Settings:** người dùng bấm xóa trực tiếp; browser gọi authenticated `DELETE /v1/memory`. Click là explicit user action, không phải model tự quyết định.

Memory side-effect serialization

Approved memory tool lấy Postgres advisory lock theo actor, verify approval, reserve idempotency theo tool-call identity, execute hoặc reuse result, commit `memory.updated`, rồi ghi execution evidence. Hai tool call đồng thời của cùng actor không silently overwrite ngoài thứ tự.

Worker boundary

Quant workers trong production dùng credential riêng `NEXUS_QUANT_WORKER_TOKEN` và `NEXUS_QUANT_DATA_WORKER_TOKEN` cùng pseudonymous actor header. AI-Trader protected endpoints dùng token riêng. Compose beta không publish backend/worker/Postgres ports; frontend chỉ bind loopback để trusted ingress đứng trước.

Container hardening

- Non-root user, drop all Linux capabilities, `no-new-privileges`.
- Read-only root filesystem; chỉ named volume/tmpfs cần thiết được writable.
- CPU, memory và PID được giới hạn theo profile private-beta 16 GB.
- Production image không chứa pip/uv/compiler/git; frontend image không chứa Node/npm.

- Runtime secret đi qua environment/secret mount, không bake vào image.
- CI scan toàn bộ Git history, audit dependency dev/runtime, tạo SBOM và chặn HIGH/CRITICAL vulnerability theo policy có kiểm soát.

Threat → control map

Threat	Control chính	Control bổ sung
Token giả/hết hạn	OIDC signature, issuer, audience, exp/iat/sub.	Asymmetric algorithm allowlist.
Cross-tenant access	HMAC actor key + PostgreSQL RLS.	DB roles và routine cross-actor tối thiểu.
Model tự publish/lưu memory	Native approval interrupt.	Digest, TTL, idempotency, execution evidence.
Prompt/event làm lộ secret	Bound input; event sensitive-key rejection.	Không ghi raw prompt/content vào product event.
Unsafe deserialization	Strict checkpoint serialization, no pickle fallback.	AES encrypted content fields.
Artifact bị sửa	Immutable manifest + SHA-256 verified download.	Status/quarantine/tombstone.
Browser render link nguy hiểm	Safe URL filtering.	CSP, nosniff, no-referrer.

15 Khởi chạy và triển khai

Yêu cầu công cụ

- Python 3.11 và `uv`.
- Node.js 22 và npm.
- PostgreSQL 16.
- Docker Compose nếu dùng container profile.

Docker quick start cho local

```
cp backend/.env.example backend/.env
cp frontend/.env.example frontend/.env
cp ai-trader-worker/.env.example ai-trader-worker/.env

# Điền model credential vào file local, không commit.
docker compose --env-file backend/.env up --build -d

# Mở:
http://localhost:8080
```

Base Compose dựng Postgres, migration job, backend, ba worker, AI-Trader refresh daemon và frontend. Backend bind loopback port 2024; AI-Trader worker có loopback port 8100 trong development; frontend mở 8080.

Host-run stack

```
cd backend && uv sync --extra dev
cd ../quant-worker && uv sync
cd ../quant-data-worker && uv sync
cd ../ai-trader-worker && uv sync --extra dev
cd ../frontend && npm install
cd ..
source backend/.venv/bin/activate
./run.sh
```

Real Qlib cần extra `qlib` của quant-worker; real OpenBB ingest cần extra `openbb` của quant-data-worker. Không có optional worker, launcher có thể bỏ qua service tương ứng, nhưng tool phải trả structured unavailable error — không được coi là end-to-end success.

Host-run ports

Service	URL	Ai gọi
Frontend Vite	<code>http://localhost:5173</code>	Browser.
LangGraph/backend	<code>http://127.0.0.1:2024</code>	Frontend dev.

Service	URL	Ai gọi
Quant worker	<code>http://127.0.0.1:8000</code>	Backend.
Quant data worker	<code>http://127.0.0.1:8001</code>	Backend.
AI-Trader worker	<code>http://127.0.0.1:8100</code>	Backend/refresh daemon.

`run.sh` làm những gì?

- 1 Tìm LangGraph CLI/Python, npm và uv.
- 2 Kiểm tra PostgreSQL; thử khởi động Compose Postgres local nếu cần; không tiếp tục khi DB không reachable.
- 3 Chạy domain migration local trừ khi `NEXUS_AUTO_MIGRATE=0`.
- 4 Kiểm tra port 2024/5173/8000/8001/8100 để tránh hai stack chồng nhau.
- 5 Khởi động worker nào có runtime, backend và frontend; ghi log theo service dưới `data/logs`.
- 6 Nếu một service dừng, terminate phần còn lại để không để stack nửa sống nửa chết.

Migration commands

```
uv run --project backend python -m source.infrastructure.migrations status
uv run --project backend python -m source.infrastructure.migrations upgrade
```

Compose dùng lệnh `bootstrap` để upgrade schema và provision roles. Rollback chỉ dành cho disposable local data với destructive flag rõ ràng; dữ liệu đã tồn tại nên dùng forward migration.

Private-beta overlay

```
docker compose + -f docker-compose.yml + -f docker-compose.beta.yml + --env-file /path/to/beta.env +
up --build -d
```

Khác biệt	Local base	Single-node private beta
Auth	Có thể dùng local actor.	OIDC bắt buộc, HTTPS issuer.
Database credential	Development default được phép.	Non-default + role riêng backend/quant-data/quant.
Worker auth	Có thể nối trong development.	Shared worker token bắt buộc.
Checkpoint	Strict serialization.	Thêm AES content encryption.
Network	Một số loopback port cho dev.	Internal network; chỉ frontend bind loopback.

Khác biệt	Local base	Single-node private beta
Volumes	Bind vào <code>./data</code> .	Named volumes cho qlib/artifact/staging/AI data.
Container	Development targets.	Production target, non-root, cap drop, tmpfs.

Không làm: không chép secret thật vào tài liệu, command history, commit, log hay screenshot.
Dùng các `.env.example` để biết tên biến và secret store/environment riêng để cấp giá trị.

16 Quan sát, lỗi và phục hồi

Health khác readiness

Probe	Trả lời câu hỏi	Ví dụ
Health	Process/API còn sống không?	Worker GET /health có process_alive=true .
Readiness	Dependency/control plane có đủ để nhận công việc không?	Worker GET /ready ; backend operational readiness kiểm tra DB.
Frontend health badge	Browser có kết nối được backend không?	Gọi /ok , cho phép refresh thủ công.

Một process có thể healthy nhưng not ready khi database, token hoặc provider dependency cần thiết chưa sẵn sàng. Autosupervision/operator không nên gộp hai tín hiệu này.

Structured logs

Các service cấu hình log có tên service, environment, event name và safe typed fields. Production default dùng JSON để tập trung log; local launcher prefix mỗi dòng theo service và tách file theo timestamped session. Không đưa secret, raw authorization hay prompt đầy đủ vào event/log.

Failure taxonomy

Failure	Dấu hiệu	Hành vi đúng
Input không hợp lệ	HTTP 422 hoặc handoff error không retryable.	Sửa symbol/date/range/model; không retry y hệt.
Worker unavailable/timeout	Stable code, thường retryable.	Giữ workflow identity và resume từ committed stage.
Manifest/hash mismatch	Validation fail hoặc HTTP 409 download.	Fail closed; không dùng bytes; điều tra/quarantine.
Approval reject/expire	Decision event và no execution reference.	Dùng side effect; không model-retry proposal.
OIDC/JWKS unavailable	401/503 auth error.	Khôi phục IdP/JWKS; không bypass production auth.
Event replay gap	gap=true .	Không giả vờ timeline đầy đủ; reload product state phù hợp.
Qlib deterministic failure	qlib_execution_failed , stage cụ thể.	Không gọi lại trừ khi error nói retryable.
AI snapshot empty	ok=true , available=false .	Refresh/configure provider; không gọi là connector failure.

Outbox projector failure

`http_app.py` chạy background loop. Nếu projection lỗi, nó log `product_event.projection_failed`, chờ poll interval và thử lại. Domain transaction đã commit không bị rollback bởi lỗi projection sau đó.

Worker restart

- Quant-data immutable revision giúp build/fetch retry không mất identity.
- Quant-worker job/lease/attempt nằm ở Postgres trong beta; coordinator recovery có thể requeue/expire theo policy.
- AI-Trader snapshot/feed nằm SQLite volume; API/refresh process tách nhau nhưng dùng cùng data volume.
- Backend checkpoint/workflow ở Postgres; browser có active run/replay cursor.

Troubleshooting theo triệu chứng

Triệu chứng	Kiểm tra hẹp nhất	Bước kế tiếp
UI “Backend ngoại tuyến”	<code>/api/ok</code> , backend process/log, port/proxy.	Kiểm tra migration/Postgres nếu backend không ready.
Graph chỉ có supervisor	<code>GET /v1/topology</code> và auth.	Kiểm tra registry/startup validation; không hardcode frontend.
Dataset build dừng ở staging	Workflow transition/error code; quant-data readiness.	Phân biệt invalid data và worker timeout.
Experiment pending lâu	Job status, progress stage, active limit/TTL.	Không submit ID mới; poll cùng experiment ID.
Artifact 410	Storage status/retention.	Dùng metadata/tombstone; bytes đã hết hạn không được giả lập.
Publish bị 409	Idempotency key/action digest.	Không reuse key cho content khác.
Memory không thay đổi	Approval status, changed flag, key normalization.	Cùng content có thể reuse/no-op hợp lệ.

Điều chưa có

Baseline không tuyên bố centralized dashboard, formal alert rules, SLO, distributed tracing certification hay off-host recovery. Product events, structured logs, probe và deterministic tests là evidence hiện có; operator phải giữ claim đúng với phạm vi đó.

17 Kiểm thử và quality gate

Local quality commands

```
# Architecture + backend
uv run --project backend python scripts/check-architecture.py
uv run --project backend python -m unittest discover -s backend/tests
uv run --project backend python -m compileall backend/source

# Workers
uv run --project quant-worker python -m unittest discover -s quant-worker/tests
uv run --project quant-data-worker python -m unittest discover -s quant-data-worker/tests
uv run --project ai-trader-worker python -m unittest discover -s ai-trader-worker/tests

# Frontend
npm --prefix frontend test
npm --prefix frontend run build
npm --prefix frontend run test:e2e

# Product validation + security
uv run --project backend python scripts/run-product-validation.py + --output data/evaluation/product-validation-v1.json
./scripts/security-audit.sh
```

Mỗi lớp chứng minh gì?

Lớp	Phạm vi chứng minh	Không tự chứng minh
Architecture check	Import boundary, registry/runtime exposure và policy drift.	Behavior của provider thật.
Backend unit	Auth, contract, workflow, approval, repositories qua doubles.	Postgres durability thực nếu không chạy integration.
Worker unit	Validation, identity, job/artifact/service behavior.	Paid/live provider availability.
Postgres integration	Migration, transaction, role/control state và recovery với DB thật.	Browser UX.
Frontend unit	Parser, reducer, replay, graph derivation, safe links.	Backend đang chạy thật.
Playwright E2E	Production browser flow và accessibility/security behavior đã mô phỏng.	Model quality hay provider quota.
Container smoke	Image import/start surface và không chứa build tool.	Full operational readiness.
Product validation	Versioned offline deterministic scenarios.	Capacity, live market, model answer quality.
Security gate	Secret/static/dependency/image vulnerability policy.	Formal penetration test.

Deterministic product-validation suite

Manifest `evaluation/product-validation-v1.json` liệt kê scenario. Scenario gắn tag `acceptance_failure` sẽ làm command fail. Report ghi status từng scenario, workflow completion và categorized failure rate. Suite offline, không gọi paid API/model.

Nhóm scenario

- Workflow ordering, validation, structured contract và fail-closed tools.
- Idempotency khi duplicate, reconnect, cancellation, worker restart.
- Tenant isolation, manifest/artifact integrity, safe evidence references.
- Point-in-time limitation và Alpha158 lookahead boundary.
- Browser replay/resume và unsafe rendered links.
- AI-Trader empty-provider semantics và input safety.

CI quality workflow

Push vào `main` và pull request kích hoạt các job backend, frontend, workers matrix, PostgreSQL integration, container build/smoke/ vulnerability, product validation và một aggregate job bắt buộc mọi nhánh đều success. Security workflow chạy thêm theo lịch hàng tuần.

Nguyên tắc đọc “green”

Green có phạm vi: nó chứng minh code hiện tại qua scenario/dependency pinned đã chạy. Nó không chứng minh provider đang online ngày mai, model luôn trả lời hay, node chịu được mọi tải, hoặc dữ liệu tài chính không còn bias.

Live beta journey vẫn cần

1. Nghiên cứu có nguồn.
2. Tạo dataset bằng provider thật.
3. Chạy experiment và xem Evidence.
4. Approval → interrupt → resume.
5. Reload/reconnect giữa run.
6. Download/export evidence và kiểm tra actor isolation.

18 Ví dụ end-to-end dễ hiểu

Ví dụ 1 — Từ câu hỏi đến Qlib experiment

Mục tiêu: tạo baseline đơn giản cho AAPL/MSFT bằng dữ liệu 2022–2024, rồi xem kết quả 2024.

Bước	Người dùng/agent	Hệ thống thực hiện	Bảng chứng sinh ra
1	“Tạo dataset AAPL, MSFT...”	Supervisor delegate quant-data-agent.	agent.delegated , typed task.
2	Quant Data gọi composite tool.	Normalize, fingerprint, fetch OpenBB.	staging revision + content hash.
3	Không cần model can thiệp.	Validate schema/OHLC và build Qlib layout.	quality checks, dataset revision, manifest.
4	Supervisor chuyển ID cho Quant Researcher.	Không đoán alias thành revision.	typed dataset reference.
5	Chọn linear baseline.	Validate segments và max_test_end; submit durable job.	workflow ID + experiment ID.
6	Agent poll cùng ID.	Worker lease/run Qlib; publish metric/artifact.	attempt, result, artifact hashes.
7	Agent giải thích limitation.	Record interpretation với exact evidence IDs.	interpretation report + lineage.
8	Người dùng mở Evidence.	Backend dựng dossier actor-scoped.	UI/export bundle.

Kết luận mẫu đúng

"Experiment exp_v1_... đã completed trên dataset dsr_v1_...
IC/return là kết quả lịch sử theo câu hình đã ghi trong dossier.
Dataset là research_only vì point-in-time membership và provider revision không được chứng minh; đây không phải lời khuyên đầu tư."

Kết luận mẫu sai

"Model này chắc chắn sẽ lời 12% năm tới."
Sai vì biến backtest thành dự báo/cam kết và bỏ qua limitations.

Ví dụ 2 — Chỉ nghiên cứu khái niệm, không chạy dữ liệu

Prompt: “IC và Rank IC khác nhau thế nào? Hãy dẫn nguồn hiện hành.”



Đúng routing vì người dùng chỉ cần lý thuyết có nguồn. Quant worker không được gọi “cho chắc”; việc tạo job thừa vừa tốn tài nguyên vừa sai scope.

Ví dụ 3 — Lưu preference có approval

Prompt: “Hãy nhớ từ nay trả lời tôi bằng tiếng Việt.”

1. Supervisor đề xuất `save_user_memory(preference.language, ...)`.
2. Middleware ghi pending approval và frontend hiện exact action.
3. Người dùng bấm Approve.
4. Graph resume; repository upsert đúng key, tăng revision nếu thay đổi.
5. `memory.updated` chỉ chứa key/revision/hash, không raw content.
6. Lướt sau middleware inject aggregate như untrusted user context.

Nếu bấm Reject: supervisor phải báo không lưu và không đề xuất lại cùng side effect để “thử sửa”.

Ví dụ 4 — Publish strategy lên AI-Trader

Prompt: “Đăng kết quả experiment vừa rồi thành strategy discussion.”

1. AI-Trader agent nhận experiment/artifact/interpretation IDs thật.
2. Nội dung gồm market/symbol, research basis, leakage/survivorship/cost/OOS limitation và câu “không phải lời khuyên đầu tư”.
3. Frontend hiện exact title/content/evidence cho người dùng duyệt.
4. Approval digest khóa đúng nội dung. Sau approve, publication service reserve idempotency và gọi worker.
5. Postgres ghi target `signal_id`; dossier nối publication với approval và evidence.

Không được: thay title/content sau approve; publish “operation” realtime; hoặc publish khi actor chỉ có `ai_trader:read`.

Ví dụ 5 — Mất mạng giữa run

1. Browser đã lưu thread ID, active run ID và last event ID trong session.
2. Mạng rớt sau khi worker accepted nhưng trước khi UI thấy `dataset.ready`.
3. Sau reload, frontend reconnect active run stream nếu còn khả dụng.
4. Frontend gọi product-event replay với `after=last_sequence`.
5. Workflow/idempotency ở Postgres ngăn submit side effect mới.

6. Nếu retention đã mất đoạn cũ, `gap=true` buộc UI không tuyên bố timeline hoàn chỉnh.

Ví dụ 6 — Provider không có snapshot

AI-Trader market-news trả `ok=true`, `available=false`, `last_updated_at=null`. Câu trả lời đúng là “worker đã kết nối nhưng chưa có cached snapshot; cần refresh và API key nếu muốn lấy dữ liệu mới”. Không nói “AI_TRADER_API_BASE_URL sai”.

19 Hướng dẫn đọc và thay đổi source

Trace một chat request

```
frontend/src/hooks/useChat.ts
  → frontend/src/api/chat.ts + chatProtocol.ts
  → backend/langgraph.json
  → backend/source/agents/agent.py
  → backend/source/bootstrap.py
  → backend/source/agents/factory.py
  → backend/source/agents/sub_agents/*
  → backend/source/tools/*
  → backend/source/application/*
  → backend/source/integrations/*
  → worker/worker/app.py
  → worker/worker/application/*
  → repository / artifact store / PostgreSQL
```

Chọn đúng nơi sửa

Muốn thay đổi	Owner chính	Consumer phải kiểm tra
Agent/task/tool topology	<code>.deepagents/nexus.json</code> , <code>prompt/skill</code> , <code>subagent module</code> .	Registry validator, frontend topology/graph, docs overview.
Supervisor behavior	<code>agents/factory.py</code> và <code>middleware</code> .	Handoff/approval/memory tests.
Quant business flow	<code>application/quant_workflows.py</code> .	Tool adapter, worker contract, workflow repository/events.
Worker HTTP contract	<code>worker/app.py</code> + <code>models</code> .	Backend integration client và tests ở hai đầu.
Domain state	<code>domain/</code> + <code>infrastructure/</code> + <code>migration</code> .	Dossier, events, worker roles, Postgres integration.
Frontend stream	<code>api/</code> , <code>useChat</code> , <code>reducer</code> .	Abort, stale guard, replay và protocol tests.
Evidence UI/API	<code>lineage_repository.py</code> , <code>http_app.py</code> , <code>DossierPage.jsx</code> .	Actor scope, export/hash, safe URL/download.
Production profile	Dockerfiles, Compose overlay, config validator.	Architecture/security/image smoke docs.

Checklist cho một thay đổi cross-layer

- 1. Đọc `AGENTS.md` gần file nhất và docs architecture.
- 2. Trace contract hai đầu trước khi sửa.
- 3. Xác định authority: model, application, DB hay worker nào được quyết định?

4. Giữ input validation ở boundary và error explicit.
5. Giữ async lifecycle, abort/stale guard và immutable React update.
6. Nếu có side effect, thiết kế approval/idempotency/retry trước happy path.
7. Thêm focused test tại lớp chịu trách nhiệm.
8. Chạy narrow test, rồi gate liên quan.
9. Cập nhật `docs/architecture/overview.md` nếu entrypoint, ownership hoặc runtime flow đổi.
10. Review diff, không stage file runtime/secret/unrelated.

Ví dụ: thêm một model Qlib mới

Đây không phải chỉ sửa một dropdown. Cần xem xét:

- Intent/worker Pydantic allowlist.
- Qlib runner implementation và dependency lock.
- Agent prompt/skill hướng dẫn khi nào được chọn.
- Structured output/config evidence.
- Unit + worker execution test + product scenario nếu là acceptance signal.
- Container size/vulnerability/runtime duration.
- Documentation workflow và PDF nếu phạm vi hỗ trợ thay đổi.

Ví dụ: thêm agent mới

Cần đồng bộ prompt file, skill file nếu có, `nexus.json`, Python subagent loader, tool/permission/budget, handoff schema, development graph, production exposure policy, topology-safe projection, frontend node/edge và architecture docs. Startup validator nên fail nếu một mặt bị bỏ sót.

Nguyên tắc: thay đổi nhỏ nhất nhưng phải coherent xuyên boundary. “Chỉ sửa chỗ compile” không đủ nếu consumer contract còn hiểu shape cũ.

20 Giới hạn hiện tại và hướng mở rộng

Giới hạn sản phẩm

Giới hạn	Tại sao quan trọng	Tín hiệu nên có trước khi mở rộng
Một node	Node/volume failure ảnh hưởng artifact và service.	Beta cho thấy nhu cầu HA/scale hoặc downtime không chấp nhận.
Filesystem artifact	Không có off-host durability/replication.	Artifact volume, retention hoặc recovery demand đo được.
Provider yfinance	Point-in-time, revision, delisting/corporate action yếu.	Use case yêu cầu production research và vendor evidence tốt hơn.
Daily + Alpha158	Không bao phủ intraday/feature families khác.	Representative users chứng minh nhu cầu cụ thể.
Bốn model + một strategy	Giảm linh hoạt nhưng tăng khả năng kiểm soát.	Comparison evidence và resource profile cho model/strategy mới.
Cached AI market intel	Freshness phụ thuộc refresh/quota.	Nhu cầu latency/freshness rõ và provider budget.
Không formal SLO	Không có claim uptime/latency định lượng.	Traffic beta, error budget và operator ownership.

Thứ tự mở rộng hợp lý

- 1 Chạy representative private-beta journeys, đo friction/failure thay vì đoán.
- 2 Biến failure thành minimal reproducible scenario và focused regression test.
- 3 Chỉ nâng scenario thành product acceptance manifest khi nó là durable signal.
- 4 Từ evidence thực quyết định remote storage, multi-node, SLO, capacity hay release provenance.

Những invariant nên giữ khi mở rộng

- Model chỉ diễn đạt ý định; trusted code sở hữu effectful workflow.
- Identity/version/hash phải đi xuyên hệ thống, không reconstruct từ prose.
- Actor scope tại mọi boundary và query.
- Approval ràng buộc exact action; retry không nhân đôi side effect.
- Evidence gap được thể hiện, không bị che bởi UI “đẹp”.
- Production fail closed khi security setting thiếu/yếu.
- Docs claim không vượt quá executable evidence.

Một câu tóm tắt: Nexus Agent hiện là một hệ thống nghiên cứu có governance và evidence cho local/private beta một node; giá trị lớn nhất của nó không phải “nhiều agent”, mà là việc nối ý định của người dùng với effect có kiểm soát và bằng chứng có thể truy vết.

21 Phụ lục A — Danh mục API

LangGraph routes frontend sử dụng

Method/path (qua /api)	Vai trò
GET /ok	Backend liveness cho browser/health check.
POST /threads	Tạo actor-owned conversation thread.
GET /threads/{thread_id}/state	Lấy graph state để resume/recover.
DELETE /threads/{thread_id}	Xóa thread của actor.
POST /threads/{thread_id}/runs/stream	Tạo run và stream nhiều event mode.
GET /threads/{thread_id}/runs/{run_id}/stream	Nối lại active run stream.
POST /threads/{thread_id}/runs/{run_id}/cancel	Yêu cầu hủy run.

Nexus product API

Method/path	Permission	Kết quả
GET /v1/runs/{run_id}/events	chat:run	Replay page sau sequence; limit 1-500.
GET /v1/topology	chat:run	Frontend-safe agent/skill/tool labels.
GET /v1/memory	chat:run	Aggregate content, revision, updated time; no-store.
DELETE /v1/memory	chat:run	Direct explicit deletion; 204.
GET /v1/operations/readiness	maintenance:run	Database operational readiness.
GET /v1/lineage/{node_id}	quant:read	Backward/forward/both graph; depth 1-8.
GET /v1/experiments/{id}/dossier	quant:read	Actor-scoped evidence dossier.
GET /v1/experiments/compare	quant:read	So sánh 2-5 experiment ID.
GET /v1/experiments/{id}/export	quant:read	JSON bundle + content-hash header.
GET /v1/artifacts/{id}/content	quant:read	Verified bounded artifact proxy.

Quant-data worker — internal

Method/path	Vai trò
GET /health , GET /ready	Process và dependency readiness.
POST /v1/openbb/ohlcv	Fetch/normalize/save OHLCV staging revision.
POST /v1/factors/snapshot	Fetch current factor snapshot revision.
POST /v1/staging/{id}/validate	Deterministic market-data quality checks.
POST /v1/qlib-datasets	Build immutable Qlib dataset.
POST /v1/qlib-datasets/{id}/validate	Validate layout/manifest/coverage.

Quant worker — internal

Method/path	Vai trò
GET /health , GET /ready	Process/dependency readiness.
GET /v1/datasets	Liệt kê dataset metadata/coverage.
POST /v1/datasets/{id}/validate?universe=...	Preflight dataset + universe.
POST /v1/experiments	Accept durable job, trả 202.
GET /v1/experiments/{id}/result	Poll status/result.
POST /v1/experiments/{id}/cancel	Cancel với reason category.
GET /v1/artifacts/{id}	Artifact manifest.
GET /v1/artifacts/{id}/content	Artifact bytes + content hash.

AI-Trader worker — internal

Method/path	Token?	Vai trò
GET /health	Không	Process health.
GET /ready	Có	Protected readiness.
GET /api/market-intel/overview	Không	Overview snapshot.
GET /api/market-intel/news	Không	Category/symbol-filtered snapshot.
POST /api/market-intel/refresh	Có	Manual provider refresh.
GET /api/signals/feed	Không	Read Nexus signal feed.
POST /api/signals/strategy	Có	Idempotent strategy publish.

Method/path	Token?	Vai trò
POST /api/signals/discussion	Có	Idempotent discussion publish.
POST /api/claw/agents/heartbeat	Có	Explicit protected poll.

22 Phụ lục B — Cấu hình quan trọng

Nhóm	Biến tiêu biểu	Ý nghĩa
Environment/model	NEXUS_ENV , MODEL , model/search credentials	Profile và provider model/tool.
OIDC	issuer, audience, JWKS URI, roles claim, allowed algorithms	Token verification và role mapping.
Actor/worker security	tenant HMAC key, quant token riêng theo service, AI-Trader token	Pseudonymous identity và internal auth.
Checkpoint	AES key, strict msgpack/content keys	Safe serialization và encryption.
PostgreSQL	host, port, user, password, database, pool min/max	Backend pool; beta có URL/role riêng theo service.
Worker URLs	quant worker, quant-data worker, AI-Trader base URL	Backend integration targets.
Timeout/poll	worker timeout, poll seconds, wait seconds	Transport và bounded poll behavior.
Input/retention	prompt max bytes, replay retention/poll, approval TTL, download max	Abuse/resource bounds.
Quant execution	active job limit, queued TTL, max execution, storage quota, artifact retention	Admission/recovery/resource policy.
Data paths	Nexus data, Qlib, staging, artifact, AI SQLite path	Runtime storage placement.
AI market refresh	provider key/base URL, lookback, limit, categories, interval, stale-after	Snapshot cadence và freshness.
Frontend runtime	OIDC authority/client/audience/scope/redirect	Browser Authorization Code flow.
Logging	log level, log format, log directory	Structured local/production observability.

Quy tắc: chỉ tra tên biến/placeholder trong `.env.example` và Compose. Không in, đọc, ghi vào PDF hay commit giá trị từ `.env` thật.

23 Phụ lục C — Glossary

Thuật ngữ	Giải thích trong Nexus
Actor	Người dùng đã xác thực, biểu diễn bằng pseudonymous HMAC key.
Agent / subagent	Model-driven role có tool/skill/scope; supervisor điều phối subagent.
Tool	Hàm được model gọi; tool cấp cao chuyển ý định sang trusted application code.
Skill	Hướng dẫn workflow chuyên môn cho agent; không tự là security boundary.
Graph	LangGraph runtime entrypoint quản lý model/tool/middleware/checkpoint.
Thread	Conversation container actor-owned.
Run	Một lần graph thực thi trong thread.
Interrupt/resume	Dừng graph tại approval và tiếp tục với quyết định của người dùng.
Checkpoint	State graph bền vững để resume/recover.
Workflow	Chuỗi nghiệp vụ xác định, versioned và durable ngoài prose của model.
Idempotency	Gửi lại cùng hành động không tạo effect thứ hai; khác args cùng key là conflict.
Fingerprint	Hash canonical của request parameters.
Content hash	SHA-256 identity của canonical content/bytes.
Revision	Phiên bản dữ liệu bất biến; revision mới không sửa revision cũ.
Manifest	Danh sách/metadata file canonical và hash dùng xác minh dataset/artifact.
Lineage	Đồ thị quan hệ giữa request/source/data/experiment/metric/artifact/claim.
Dossier	Hồ sơ evidence hợp nhất của một experiment.
Transactional outbox	Ghi event intent cùng domain transaction, project sang replay sau.
Point-in-time	Dữ liệu phản ánh đúng thứ đã biết ở thời điểm lịch sử; baseline provider chưa chứng minh.
Survivorship bias	Thiên lệch khi universe chỉ gồm tài sản còn tồn tại/được biết hiện tại.
Lookahead leakage	Dùng thông tin tương lai khi train/test; Alpha158 label cần chờ hai ngày.
OHLCV	Open, High, Low, Close, Volume.
Alpha158	Feature/label handler Qlib được allowlist trong MVP.
TopkDropoutStrategy	Strategy chọn top K và thay một số vị trí theo signal.
Evidence gap	Phần bằng chứng thiếu/yếu được biểu diễn rõ thay vì che giấu.

Thuật ngữ	Giải thích trong Nexus
Fail closed	Khi không xác minh được thì từ chối/giảm quyền, không đoán là an toàn.

24 Phụ lục D — Checklist vận hành

Trước khi chạy local

- Đúng Python 3.11, Node 22, uv/npm và Postgres 16.
- Tạo file local từ `.env.example`; không commit.
- Cài extra Qlib/OpenBB nếu cần provider/experiment thật.
- Kiểm tra port không bị stack cũ chiếm.
- Chạy migration status/upgrade.

Trước khi chạy private beta

- OIDC issuer/audience/client/redirect đúng HTTPS.
- Database password non-default và role riêng từng service.
- Tenant HMAC, worker token, AI-Trader token và AES key đủ mạnh.
- Chỉ frontend bind loopback/trusted ingress; worker không public.
- Named volumes có ownership/backup policy phù hợp với claim một node.
- Quality, security, container vulnerability và product validation đều green.
- Chạy live journey gồm approval, reconnect, Evidence và actor isolation.

Khi đánh giá một kết quả quant

- Dataset revision và manifest verified?
- Train/valid/test đúng thứ tự và nằm trong coverage/max_test_end?
- Metric có definition/calculation version/evidence reference?
- Artifact hash/status/download eligibility hợp lệ?
- Point-in-time, survivorship, adjustment, calendar, cost và OOS limitation đã nêu?
- Interpretation có phân biệt historical evidence và investment advice?

Khi xử lý một side effect

- Actor có permission?
- Exact arguments đã normalize và hiện cho người dùng?
- Approval còn hiệu lực và action digest khớp?
- Idempotency key có gắn đúng action, không reuse cho content khác?
- Execution result/reference đã ghi; failure có safe code/stage/retryable?

Khi cập nhật tài liệu PDF này

1. Đổi source baseline/date trên trang bìa.
2. Đối chiếu endpoint, topology, route, permission, state machine và config.
3. Cập nhật HTML, tạo lại PDF bằng lệnh trong `docs/pdf/README.md`.
4. Kiểm tra PDF mở được, font tiếng Việt, mục lục/link, bảng không bị cắt.

5. Review diff và bảo đảm không có secret/runtime data.

Kết thúc: Nếu chỉ nhớ một điều, hãy nhớ rằng Nexus Agent tách *lời nói của model* khỏi *quyền thực hiện hành động*. Contract, workflow, approval, identity, persistence và evidence là những lớp biến một demo nhiều agent thành một sản phẩm có thể kiểm tra.